Trabajo de Taller IISS 2026 — Iteración 2

Cierre de alcance y persistencia

Contexto / Objetivo

Cerrar una primera definición del alcance del producto e incorporar la persistencia de la configuración de habitaciones del sitio, partiendo del prototipo de mensajería MQTT construido en la Iteración 1.

Alcance por plano

Diseño del Producto

La empresa ha optado por la siguiente Visión del Producto:

> Para hogares y oficinas con calefacción por losa radiante que buscan maximizar su confort y también optimizar su consumo eléctrico, el software **IoTEste EcoWarm** es un componente de una solución de domótica que realiza la gestión inteligente de sensores y switches; a diferencia de otras soluciones de domótica que sólo automatizan algunas acciones, nuestro producto realiza la gestión del consumo eléctrico en forma inteligente optimizando el consumo según las tarifas disponibles en nuestro mercado.

A partir de esa visión, el equipo debe:

- Refinar y actualizar la Visión del Producto elaborada en la Iteración 1.
- Describir:
  - 3 **Escenarios** donde es interesante analizar el funcionamiento del producto
  - 3 **Historias** de usuario vinculadas a los escenarios, que les parecen relevantes para describir características importantes de la propuesta.
  - 5 **Características** del producto

Algunos elementos a tener en cuenta para los escenarios, historias y características:

- Los clientes van a utilizar tarifa de UTE multihorario; hay dos alternativas que deben ser consideradas.
- En cada habitación va a existir 1 termostato y 1 switch para controlar el funcionamiento del sistema de calefacción.
- Cada habitación va a tener una temperatura objetivo expresada en grados celsius.
- Podrían existir sensores exteriores de temperatura/humedad.
- Hay posibilidad de acceder a pronósticos de tiempo de las próximas 48 hs.
- Interesa recabar información de la gestión pasada y evaluar los resultados. Por ejemplo, cuánto tarda en calentar una habitación y cuánto tarda en enfriarse.

- Se quiere disponer de capacidad de simulación, para que el cliente pueda conocer, con distintas potencias contratadas, qué temperatura es posible lograr en distintas configuraciones (cuántas habitaciones deben estar encendidas).
- Sería una ventaja adicional si se puede usar GenAI para 'dialogar' con el sistema o aprovechar las capacidades de un LLM.

Durante el plazo de la iteración se podrán realizar consultas a potenciales clientes e interesados.

Herramientas y Técnicas

- Adoptar y **declarar por escrito** una metodología ágil (Scrum, Kanban o una combinación), justificando la elección, y gestionarla en **Jira**: un Board con al menos las columnas `Todo`, `InProgress`, `ToTest`, `Done`, y un Backlog con **Épicas** para las historias de alcance mayor (documento docs/metodologia.md)
- Adoptar **Maven** para compilación, ejecución de tests y empaquetado del sistema.
- El **build debe ser reproducible sin depender de herramientas instaladas en el sistema** (Java, Maven, etc.): se realiza **mediante Docker**, a través de un script `scripts/build.sh` que compila todo el sistema.
- Configurar **GitHub Actions** con Maven para la integración continua (ver: https://docs.github.com/en/actions/tutorials/build-and-test-code/java-with-maven).

El equipo aplicará la metodología ágil a lo largo del proyecto; al final de cada iteración se revisará su cumplimiento. Se podrá ajustar la metodología cuando se justifique, declarando los cambios que se incorporan.

Prototipo

Si bien el producto a desarrollar aún está en elaboración, se debe extender el prototipo de la Iteración 1 para ir incorporando características que se estima serán la base del producto.

El prototipo debe gestionar internamente el conjunto de habitaciones donde se va a controlar la temperatura: - El prototipo modela un **sitio** que consiste en un conjunto de **habitaciones** (o *rooms*). Cada habitación tiene asociados **un termostato, un switch y una temperatura esperada** (celsius con 1 decimal). El termostato es el dispositivo que mide la temperatura de la habitación (un sensor Shelly H&T). - Se debe mantener de forma **persistente** la lista de habitaciones controladas, identificando para cada una su termostato, su switch y su temperatura esperada.

Generador de Eventos: Se debe incluir un generador de eventos desarrollado en Java, como un componente más del sistema (integrado como un contenedor al `docker-compose`): - **El generador de eventos de termostatos (datos de prueba):** cliente Java que publica por MQTT eventos de temperatura, simulando el envío de los termostatos (sensores Shelly H&T) (estructura del mensaje según el ejemplo adjunto). Es un generador de datos propio del equipo.

Consumidor de Eventos: Extensión del consumidor de eventos: - **Cliente receptor:** es extendido, ahora recibe los eventos de temperatura, los **despliega en la consola** (salida estándar) y además los **persiste**. Para cada habitación lo persistido lleva registro por fecha y hora de la temperatura.

Cuando se levanta el docker compose el sistema se pone a correr, el generador produce eventos, el consumidor reporta en consola y persiste, todo esto para una configuración predefinida.

Entregables de la iteración

- Documento de **Visión v2**.
- Los **Escenarios, Historias y Características** de EcoWarm, ver arriba cuántos como mínimo.
- Metodología ágil declarada y justificada (docs/metodologia.md), con el **proyecto Jira** configurado (Board + Backlog con Épicas/Historias al día).
- El **Prototipo v2** desplegable mediante `docker-compose` con: generador de eventos de termostatos (Java) + cliente receptor (Java) que **despliega los eventos en consola** y un componente **repositorio** para persistir los registros de temperatura y datos del sitio.
- **Integración continua** con GitHub Actions + Maven operativa.

Mecanismo de entrega

- Tag `version2` en GitHub. Copia `.zip` en Moodle.
- Archivos clave:
  - `/docs/vision.md` (v2)
  - `/docs/producto/escenarios.md`
  - `/docs/producto/historias.md`
  - `/docs/producto/caracteristicas.md`
  - `/docker/docker-compose.yml` (actualizado con la persistencia y los dos clientes Java)
  - `/src/...` (los dos clientes Java, como módulos Maven)
  - `/scripts/build.sh` (compila todo el sistema mediante Docker; no debe depender de herramientas instaladas en el sistema)
  - `/.github/workflows/maven.yml`
  - `/README.md` (**actualizado a esta iteración**, entre otros documentos y artefactos que deban reflejar el avance)

Material adjunto

- El mensaje que vamos a usar para representar la temperatura es:
  - Topic: ht-sim-room1/status/temperature:0
  - `{"id":0,"tC":21.4,"tF":70.5,"ts":1786840680.0}`
  - ts = epoch en segundos, con decimales de milisegundos. `ts = epochMillis / 1000.0 → 1786840680.123`
  - tC: temperatura Celsius
  - tF: temperatura Fahrenheit